Auditoría de Arquitectura y Corrección de Errores - Subsistema Reciminsa Ops & Field

Este documento recopila los 15 fallos de diseño, base de datos, sincronización y seguridad ("chivos") detectados en la especificación técnica inicial del subsistema Reciminsa Ops y Reciminsa Field, junto con la solución de ingeniería recomendada para cada uno antes de pasar a producción.

Bloque 1: Errores de Base de Datos, Sincronización e Idempotencia

1. El "Huevo y la Gallina" en dispositivos_campo (Error de Base de Datos)

- **Problema Detectado**: En el esquema SQL, la tabla `dispositivos_campo` exige `android_id VARCHAR(64) UNIQUE NOT NULL` y en esa misma tabla almacena `pairing_code` y `pairing_expires_at`. Sin embargo, el flujo de negocio dicta que el administrador genera el código en Reciminsa Ops antes de que el empleado abra la aplicación por primera vez. Como en ese instante aún se desconoce el `android_id` del teléfono, PostgreSQL arrojará un error `NOT NULL constraint failed` al intentar insertar el código de vinculación.
- **Solución Técnica**:
    - Mover las columnas `pairing_code` y `pairing_expires_at` a la tabla `colaboradores` (o crear una tabla efímera `codigos_vinculacion`), o bien permitir que `android_id` y `device_model` acepten valores `NULL` inicialmente hasta que el dispositivo reclame el código en `POST /api/v1/device/pair`.

2. Falta de Aislamiento Multi-Empresa (empresa_id)

- **Problema Detectado**: La especificación indica que el empleado sube bitácoras respetando las reglas de las empresas, pero ninguna tabla del esquema SQL (`colaboradores`, `clientes`, `materiales`, `bitacora_recogida`) incluye la clave foránea `empresa_id`. Si el endpoint `POST /api/v1/auth/sync` devuelve los

catálogos sin filtrar por empresa, un chofer descargará clientes y materiales pertenecientes a otras razones sociales o sucursales.

- **Solución Técnica**:
  - Añadir `empresa_id BIGINT NOT NULL REFERENCES empresas(id)` en `colaboradores`, `materiales`, `clientes` y `bitacora_recogida`.
  - Filtrar estrictamente todas las consultas de sincronización (`auth/sync`) e inserciones de bitácora extrayendo el `empresa_id` directamente del token/colaborador autenticado en el backend.

3. Registros Duplicados al Recuperar Señal (Falta de Idempotencia en bitacora_recogida)

- **Problema Detectado**: En zonas de baja cobertura, el worker local (`WorkManager`) reintentará vaciar la cola de SQLite al recuperar internet. Como `bitacora_recogida` solo tiene un `id BIGSERIAL` autogenerado en el servidor, si el celular envía un pesaje, el servidor lo inserta, pero la conexión cae en el milisegundo previo a que el celular reciba el `200 OK`, la app asumirá que falló y volverá a enviarlo, duplicando kilos y costos en el almacén.
- **Solución Técnica**:
  - Añadir la columna `client_record_uuid UUID UNIQUE NOT NULL` en `bitacora_recogida`.
  - La app móvil genera un UUID v4 único en el instante en que el recolector guarda el pesaje en SQLite.
  - El backend ejecuta la inserción con `ON CONFLICT (client_record_uuid) DO NOTHING`, garantizando idempotencia total aunque el lote se reenvíe múltiples veces.

4. Desfase de Reloj entre el Celular y el Servidor (Falsos Desconectados)

- **Problema Detectado**: El payload de telemetría envía el `timestamp` del reloj interno del teléfono, y la regla de desconexión compara `Tiempo_Servidor - Tiempo_Ultima_Posicion <= 60 segundos`. Si el teléfono del chofer tiene el reloj manual atrasado o adelantado por apenas 65 segundos, aparecerá permanentemente como "Desconectado / Sin señal" en el mapa de Reciminsa Ops aunque esté transmitiendo cada 10 segundos sin fallos.
- **Solución Técnica**:
  - Separar el tiempo de captura satelital (`timestamp_gps`) del tiempo de recepción en servidor (`last_seen_at`).

    - Al recibir un paquete en `POST /api/v1/telemetry/location`, el servidor debe actualizar `dispositivos_campo.last_seen_at = CURRENT_TIMESTAMP` usando exclusivamente el reloj del servidor.
    - La regla de los 60 segundos debe evaluarse como `CURRENT_TIMESTAMP - last_seen_at <= INTERVAL '60 seconds'`.

5. Catálogos Congelados Después del Primer Día

- **Problema Detectado**: El flujo establece que el dispositivo se vincula una sola vez sin reautenticación futura y descarga los clientes y materiales al conectarse. Si la oficina da de alta un cliente nuevo o un nuevo tipo de material días después del emparejamiento, el teléfono del empleado nunca recibirá esa actualización si el endpoint solo se invoca durante el pairing.
- **Solución Técnica**:
    - Ejecutar `POST /api/v1/auth/sync` automáticamente en cada inicio de la aplicación Reciminsa Field (si hay red disponible) y antes de iniciar jornada.
    - Incluir un botón manual de "Sincronizar Catálogos" en la interfaz de campo y soportar sincronización diferencial mediante un parámetro opcional `updated_since`.

6. Redundancia de Credenciales entre X-Device-Token y api_key

- **Problema Detectado**: Los ejemplos de la especificación muestran simultáneamente un header HTTP `X-Device-Token: eyJ...` (formato JWT) y un campo `"api_key": "sec_dev_..."` distinto dentro del cuerpo JSON. Esto introduce ambigüedad sobre cuál credencial valida el middleware de seguridad.
- **Solución Técnica**:
    - Estandarizar la autenticación en un único mecanismo: enviar el `api_key` en el header estándar `Authorization: Bearer <api_key>` (o `X-Api-Key`) acompañado del header `X-Android-Id`, eliminando tokens duplicados del cuerpo JSON.

7. Falta de Contrato JSON en POST /api/v1/bitacora/entry y Soporte de Lotes

- **Problema Detectado**: Se menciona el endpoint `POST /api/v1/bitacora/entry` y el vaciado por lotes desde SQLite, pero no se define la estructura del payload JSON ni si acepta un arreglo de registros para sincronización offline masiva.
- **Solución Técnica**:

- Definir el contrato del endpoint aceptando un arreglo `entries` para procesar tanto envíos individuales en línea como colas pendientes offline en una sola transacción:

n
POST /api/v1/bitacora/entry
Authorization: Bearer sec_dev_9941a0b34d7281
X-Android-Id: 4b2e8d901a3f6c12

{
"entries": [
{
"client_record_uuid": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
"material_id": 2,
"cliente_origen_id": 101,
"peso_kg": 145.50,
"latitud_captura": 18.486058,
"longitud_captura": -69.931212,
"notas": "Pacas limpias",
"timestamp_captura": "2026-11-03T14:20:00Z"
}
]
}

None

```
## Bloque 2: Errores Operativos, Integridad de Hardware y Rendimiento

### 8. Imposibilidad de Reemplazar Teléfonos Rotos o Revocar Accesos (ON DELETE RESTRICT)

* **Problema Detectado**: La tabla `bitacora_recogida` tiene `dispositivo_id BIGINT NOT NULL REFERENCES dispositivos_campo(id) ON DELETE RESTRICT`, y `dispositivos_campo` carece de una columna de estado (`activo`). Si un empleado registra bitácoras y meses después rompe el teléfono, lo pierde o es desvinculado de la empresa:
  * No se puede eliminar la fila en `dispositivos_campo` porque PostgreSQL lo bloquea para proteger el historial de bitácoras.
  * No existe un campo booleano en `dispositivos_campo` para revocar ese `api_token_hash` sin borrar el registro.
```

```
* **Solución Técnica**:
  * Agregar las columnas `activo BOOLEAN NOT NULL DEFAULT TRUE` y `revoked_at TIMESTAMP WITH TIME ZONE` en `dispositivos_campo`.
  * En el middleware de la API, rechazar cualquier petición con código `403 Forbidden` si `dispositivos_campo.activo = FALSE` o `colaboradores.estado_activo = FALSE`, permitiendo a Reciminsa Ops bloquear un teléfono robado al instante sin perder trazabilidad histórica.

### 9. Bloqueo al Reinstalar la App por Restricción UNIQUE en android_id

* **Problema Detectado**: El token se guarda en `EncryptedSharedPreferences` en el teléfono, mientras que `android_id` tiene restricción `UNIQUE NOT NULL` en la base de datos. Si el empleado desinstala la app o borra datos por error, el teléfono pierde el token local pero conserva exactamente el mismo `Settings.Secure.ANDROID_ID`. Al generarle un nuevo código de vinculación e intentar registrarlo, el `INSERT` en `dispositivos_campo` fallará con un error de clave duplicada (`duplicate key value violates unique constraint`).
* **Solución Técnica**:
  * Implementar una operación `UPSERT` en el endpoint de vinculación (`ON CONFLICT (android_id) DO UPDATE SET colaborador_id = EXCLUDED.colaborador_id, api_token_hash = EXCLUDED.api_token_hash, activo = TRUE, revoked_at = NULL`). Así, si el mismo dispositivo físico se vuelve a emparejar con un código válido, se actualiza su token limpiamente sin romper la restricción `UNIQUE`.

### 10. Inconsistencia entre el JSON de Sincronización y las Tablas SQL (materiales y clientes)

* **Problema Detectado**: En la respuesta JSON de `POST /api/v1/auth/sync`, el arreglo `materiales` incluye `"precio_referencia": 5.50` y `"unidad": "kg"`, y `clientes` incluye `"contacto": "Juan Pérez - 809-555-1234"`. Sin embargo, en el script SQL de la Sección 6:
  * La tabla `materiales` no tiene columna `precio_referencia` y la columna de unidad se llama `unidad_medida`.
  * La tabla `clientes` no tiene columna `contacto` (solo `telefono`).
  * Además, enviar precios a la app de campo expone información contable sensible si la labor del recolector es únicamente pesar material.
* **Solución Técnica**:
  * Homologar los nombres de campos entre las tablas SQL (`unidad_medida`, `contacto_nombre`, `telefono`) y el serializador JSON de la API.
  * Omitir `precio_referencia` del payload enviado a Reciminsa Field a menos que el recolector realice compras en efectivo autorizadas en ruta.
```

11. Coordenadas de Pesaje Prometidas pero Inexistentes en bitacora_recogida

* **Problema Detectado**: La especificación de resiliencia offline indica que las lecturas de pesaje y sus coordenadas asociadas se almacenan localmente, pero la definición SQL de la tabla `bitacora_recogida` no posee columnas de latitud ni longitud. Esto impide auditar en oficina si el pesaje fue registrado realmente en el local del cliente o a kilómetros de distancia.
* **Solución Técnica**:
  * Añadir `latitud_captura NUMERIC(10, 7)` y `longitud_captura NUMERIC(10, 7)` (más `precision_gps NUMERIC(6, 2)`) directamente en la tabla `bitacora_recogida`.

12. Explosión de Almacenamiento en tracking_ubicaciones (Millones de Filas por Mes)

* **Problema Detectado**: Transmitir coordenadas cada 10 segundos e insertar cada petición en la tabla histórica `tracking_ubicaciones` genera 8,640 filas por día por cada empleado (incluso con el camión apagado o estacionado en el almacén). Con una flota de 10 choferes se acumularán más de 2.5 millones de registros mensuales, degradando severamente los índices y el espacio en disco.
* **Solución Técnica**:
  * En cada recepción de 10 segundos, ejecutar siempre el `UPDATE` sobre `dispositivos_campo` (`last_latitude`, `last_longitude`, `last_seen_at`) para mantener el marcador de oficina en verde y en tiempo real.
  * Condicionar el `INSERT` en la tabla histórica `tracking_ubicaciones` únicamente cuando la distancia entre la nueva coordenada y la última guardada supere los 15 metros (fórmula Haversine) o hayan pasado más de 5 minutos en reposo.
  * Configurar una tarea programada de limpieza o particionado mensual para archivar o purgar el historial detallado mayor a 60 días.

13. Rastreo Continuo 24/7 Fuera de Horario Laboral

* **Problema Detectado**: Dado que el dispositivo no cierra sesión nunca y el `Foreground Service` transmite cada 10 segundos de forma ininterrumpida, la aplicación seguirá enviando la ubicación del trabajador cuando esté en su residencia por la noche o en días libres, agotando su batería, consumiendo datos móviles y generando fricciones de privacidad.
* **Solución Técnica**:
  * Incorporar en Reciminsa Field un control de estado de turno ("Iniciar Jornada / Finalizar Jornada") o parametrizar en Reciminsa Ops las ventanas de horario

laboral (por ejemplo, Lunes a Sábado de 7:00 AM a 6:00 PM) para que el servicio detenga el muestreo GPS fuera de turno.

14. Vulnerabilidad ante Ubicaciones Falsas (Mock GPS) y GPS Desactivado

* **Problema Detectado**: El payload de telemetría confía ciegamente en la latitud y longitud enviadas por el cliente sin verificar si el usuario activó aplicaciones de suplantación de GPS (*Mock Locations*) en Android o si desactivó el sensor de ubicación (enviando coordenadas congeladas en caché mientras sigue teniendo internet).
* **Solución Técnica**:
  * En Android, inspeccionar cada lectura con `location.isMock()` (API 31+) o `location.isFromMockProvider()`.
  * Incluir en el payload de telemetría los flags `"is_mock": false` y `"gps_enabled": true`. Si `is_mock` es verdadero, alertar visualmente en el mapa de Reciminsa Ops y bloquear el registro de bitácoras hasta que desactive el simulador.

15. Tabla de Especificación Incompleta en Telemetría

* **Problema Detectado**: El ejemplo JSON de `POST /api/v1/telemetry/location` incluye las propiedades `speed` y `accuracy`, pero la tabla descriptiva de campos debajo del bloque de código las omite, dejando sin definir sus unidades (metros por segundo y radio de precisión en metros).
* **Solución Técnica**:
  * Documentar formalmente en la tabla de contrato los campos `speed` (Float, opcional, velocidad en m/s), `accuracy` (Float, requerido, radio de incertidumbre horizontal en metros), `is_mock` (Boolean) y `battery_level` (Integer, opcional, porcentaje de batería 0-100 para distinguir entre un apagado por batería agotada y un apagado intencional).

Anexo: Esquema SQL Corregido y Blindado

```sql
-- Catálogo central de colaboradores operativos
CREATE TABLE colaboradores (
    id BIGSERIAL PRIMARY KEY,
    empresa_id BIGINT NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    estado_activo BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    pairing_code VARCHAR(32),
    pairing_expires_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Catálogos maestros sincronizados hacia dispositivos de campo
CREATE TABLE materiales (
    id BIGSERIAL PRIMARY KEY,
    empresa_id BIGINT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    categoria VARCHAR(50),
    unidad_medida VARCHAR(20) NOT NULL DEFAULT 'kg',
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE clientes (
    id BIGSERIAL PRIMARY KEY,
    empresa_id BIGINT NOT NULL,
    nombre_comercial VARCHAR(150) NOT NULL,
    rnc_cedula VARCHAR(20),
    direccion TEXT,
    contacto_nombre VARCHAR(100),
    telefono VARCHAR(20),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Dispositivos móviles autorizados y estado de enlace en tiempo real
CREATE TABLE dispositivos_campo (
    id BIGSERIAL PRIMARY KEY,
    colaborador_id BIGINT NOT NULL REFERENCES colaboradores(id) ON DELETE
RESTRICT,
    android_id VARCHAR(64) UNIQUE NOT NULL,
    device_model VARCHAR(100),
    api_token_hash VARCHAR(128) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    revoked_at TIMESTAMP WITH TIME ZONE,
    last_latitude NUMERIC(10, 7),
```

```
    last_longitude NUMERIC(10, 7),
    last_accuracy NUMERIC(6, 2),
    is_mock_location BOOLEAN DEFAULT FALSE,
    battery_level SMALLINT,
    last_seen_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Registro histórico de telemetría (insertar solo si hay desplazamiento > 15m)
CREATE TABLE tracking_ubicaciones (
    id BIGSERIAL PRIMARY KEY,
    dispositivo_id BIGINT NOT NULL REFERENCES dispositivos_campo(id) ON DELETE
CASCADE,
    latitud NUMERIC(10, 7) NOT NULL,
    longitud NUMERIC(10, 7) NOT NULL,
    velocidad NUMERIC(6, 2),
    precision_gps NUMERIC(6, 2),
    is_mock BOOLEAN DEFAULT FALSE,
    timestamp_gps TIMESTAMP WITH TIME ZONE NOT NULL,
    received_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Transacciones de recolección de material (Inmutable e Idempotente)
CREATE TABLE bitacora_recogida (
    id BIGSERIAL PRIMARY KEY,
    client_record_uuid UUID UNIQUE NOT NULL,
    empresa_id BIGINT NOT NULL,
    colaborador_id BIGINT NOT NULL REFERENCES colaboradores(id) ON DELETE
RESTRICT,
    dispositivo_id BIGINT NOT NULL REFERENCES dispositivos_campo(id) ON DELETE
RESTRICT,
    material_id BIGINT NOT NULL REFERENCES materiales(id) ON DELETE RESTRICT,
    cliente_origen_id BIGINT NOT NULL REFERENCES clientes(id) ON DELETE RESTRICT,
    peso_kg NUMERIC(10, 2) NOT NULL CHECK (peso_kg > 0),
    latitud_captura NUMERIC(10, 7),
    longitud_captura NUMERIC(10, 7),
    precision_gps NUMERIC(6, 2),
    notas TEXT,
    timestamp_captura TIMESTAMP WITH TIME ZONE NOT NULL,
    sincronizado_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Índices de rendimiento
CREATE INDEX idx_dispositivos_campo_last_seen ON dispositivos_campo(last_seen_at)
WHERE activo = TRUE;
CREATE INDEX idx_tracking_ubicaciones_disp_ts ON
tracking_ubicaciones(dispositivo_id, received_at DESC);
CREATE INDEX idx_bitacora_recogida_empresa_ts ON bitacora_recogida(empresa_id,
colaborador_id, timestamp_captura DESC);
```